个人作品集 · 工作流专题

小红书自动化发布系统
工作流设计 • 设计目的 • Skill 改良
（脚本代码均为vibe coding）

一个把「选题 → 文案 → 配图 → 发布 → 数据回收」全链路自动化的工作流，以及为跑通它所做的工程改良与取舍。

作者：**付思源**　　角色：**工作流设计 & 工程实现**　　周期：**2026.06 — 至今**

核心：**LLM + MCP + CDP**

01 设计目的

做这套系统，不是"为了自动化而自动化"，而是想解决内容运营里一个很具体的矛盾：**重复劳动挤占了真正有价值的思考时间**。所以设计目标从一开始就很明确，

GOAL 01

把重复动作交给机器

想标题、写正文、配图、打标签、发出去——这些"每天都要重做一遍"的动作，应被流程化，人来定策略、机器来执行。

GOAL 02

让内容质量可稳定复现

靠模板 + 模型约束，把"看状态发挥"的写作，变成"照模板产出"的稳定输出，减少波动。

GOAL 03

能力可被 Agent 调用

把发布封装成标准接口（MCP 工具），让 AI Agent 能"发现并调用"，而不只是人能手动跑脚本。

GOAL 04

流程能闭环、可沉淀

发布不是终点。数据要能回收、踩坑要能沉淀，下一轮选题才有依据，

三条设计原则

分层解耦

决策、接口、执行三层分开，任一层可独立替换与调试。

失败要可见

宁可显性报错，也不静默失败；绝不复"看起来成功"的假动作，

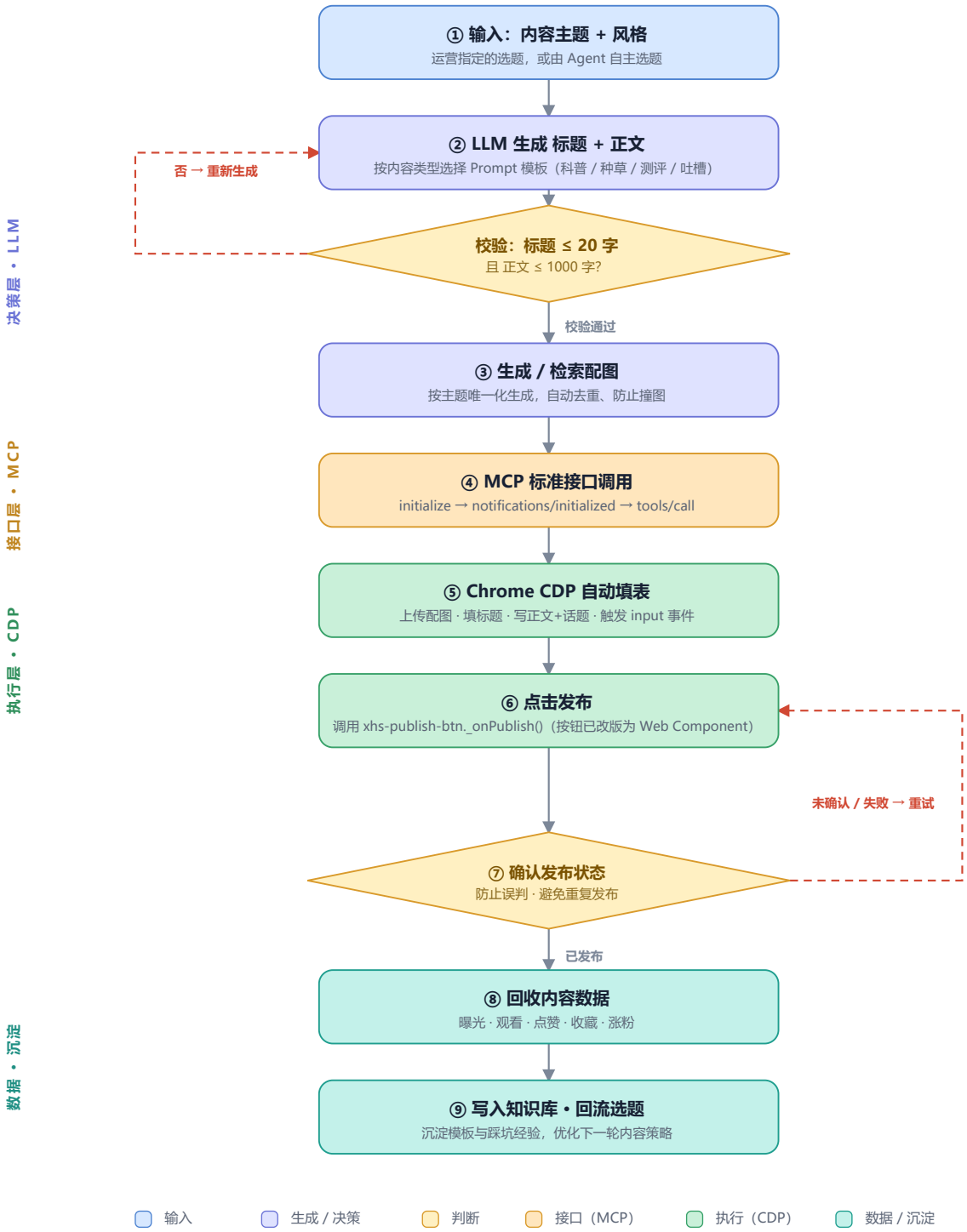
风险有边界

正视平台风控，默认小流量、人工复核，不无脑刷量，

> 一句话：这套工作流的目标，是把"体力活"压缩掉，把运营的价值重新交还给**判断与策略**。

小红书自动化发布 · 全流程工作流

从「一个主题」到「一篇笔记 + 数据回收」的端到端自动化链路 · Agent 决策层 / MCP 接口层 / CDP 执行层



红色虚线 = 反馈回路（校验失败重写 / 发布未确认重试）

03 Skill 改良：前后对比

工作流能真正跑通，靠的是一轮轮踩坑与改良。下面每条都是**真实遇到并修掉**的问题，采用「改良前 vs 改良后」直观看差异。

1 发布按钮改版：选择器失效 → 调用组件方法

● 改良前

依靠固定 CSS 选择器点按钮：

```
# 选择器硬编码
el = qs('.publish-page-publish-btn button.bg-red')
el.click()
# 平台改版后 → 找不到元素，发布失败
```

● 改良后

优先调用组件实例方法，再回退点击：

```
# 适配新版 Web Component
el = qs('xhs-publish-btn')
if el and hasattr(el,'_onPublis
    el._onPublish()
else:
    fallback_mouse_click()
```

> **效果：**适配 2026.06 平台改版，发布动作重新可用。

2 富文本编辑器静默不识别：直接改 DOM → 补触发事件

● 改良前

把正文直接写进编辑器就以为完成，结果发布按钮一直灰着：

```
editor.innerHTML = content
# ProseMirror 未收到变更事件
# → 按钮 20s 内不激活，报错
```

● 改良后

写入后主动派发 `input` 事件，让编辑器感知变更：

```
editor.innerHTML = content
editor.dispatchEvent(
  new Event('input',{bubbles:true}))
# → 按钮激活，可发布
```

> **效果：**解决 `Publish button did not become ready` 的根因。

3 隐藏的平台规则：无校验 → 发布前强制校验

● 改良前

标题超字数直接发失败，且**没有任何报错提示**，只能靠猜：

- 标题超过限制 → 静默失败
- 正文超 1000 字 → 静默失败
- 文档与 Skill 均未标注此规则

● 改良后

把平台硬规则前置成校验（中文/中文标点按 2 计）：

```
if len_weight(title) > 20:       # 自动截断
if len_weight(content) > 1000:     # 自动精简/重写
```

效果：把"看不见的失败"变成"发布前可拦截"。

4 MCP 通信细节：协议不匹配 → 按新规范对接

● 改良前

```
# 旧版 stdio 帧 → 新版 SDK 超时
Content-Length: 137\r\n\r\n{ ... }
# 通知里带 id → 被拒绝
{jsonrpc,id:1,method:'notifications/initialized'}
# 缺少 Accept 头 → 拿不到响应
```

● 改良后

```
# 换行分隔 JSON，无 Content-Lengt
{ "jsonrpc":"2.0","method":" ... /
# 通知不带 id（hasId=false）
headers['Accept'] = 'application
```

效果：MCP Server 探活通过，Agent 侧调用不再"失明"。

5 失败误判：看退出码 → 先核实真实状态

● 改良前

脚本因日志编码（GBK 不支持 emoji）抛错、退出码非 0：

- 被判定为"发布失败"
- 触发重试 → **内容被重复发布**

● 改良后

建立"先确认、再决定"的纪律：

- 先回收真实发布状态
- 仅确认失败才重试
- 日志编码问题单独处理

效果：杜绝重复发布，脚本"报错"不再等于"没发出去"。

6 配图：复用旧图 → 按主题唯一化

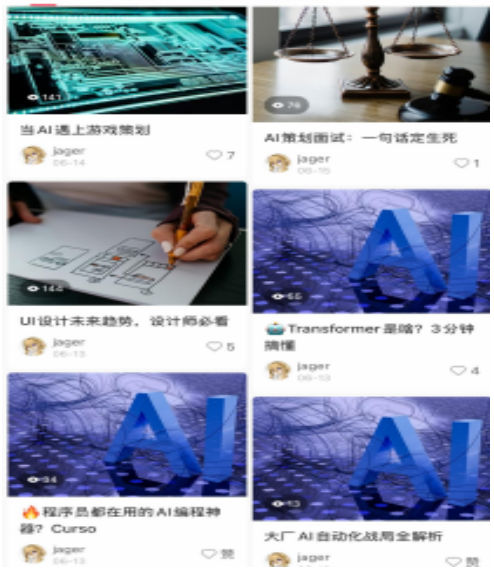
● 改良前

- 多篇内容复用同一批图
- 容易撞图、与主题不符

● 改良后

- 每个主题独立生成 / 检索配图
- 入库去重，避免重复使用

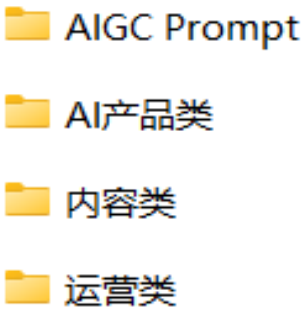
效果：内容观感更专业，降低同质化风险。





改良前

改良后



文案模版

```
## 必做约束

- 需按照C:\Users\Administrator\.openclaw\workspace\prompt-library里对应的prompt模版进行编写文案（标题不做要求）*如果内容是产品类请检索“AI产品类”的对应Prompt模版，内容是旅游指南或者种草推荐类请检索“内容类”的对应Prompt模版，内容是分析经验类请检索“运营类”的对应Prompt模版*
- 发布前必须让用户确认最终标题、正文和图片/视频。
- 图文发布时，没有图片不得发布（小红书发图文必须有图片）*不能使用重复的图片，先使用C:\Users\Administrator\.openclaw\workspace\skills\AIGC\SKILL.md,如果无法生成需使用‘web-search‘搜索用户要求的内容*。
- 视频发布时，没有视频不得发布。图片和视频不可混合使用（二选一）。
- 默认使用无头模式；若检测到未登录，切换有窗口模式登录。
- 标题长度一定不能超过 20*！！否则就算执行完所有程序也无法发布！！*（中文/中文标点按 2，英文数字按 1）。
- 用户要求"仅测试浏览器"时，不得触发发布命令。
- 如使用文件路径，优先使用绝对路径；若用户给的是相对路径，先转换为绝对路径再执行命令。
- 若发布页结构异常，优先检查 `scripts/cdp_publish.py` 里的 `SELECTORS`、多图上传等待、正文编辑器与发布按钮点击逻辑；这些是最容易被小红书网页改版影响的区域。
```

SKILL优化

这六条改良的共同点：**不是"功能加了多少"，而是"把不稳定的地方变得稳定"**——平台会改版、协议会演进、错误会伪装，能持续跑通，靠的是对失败原因的深挖与工程化的兜底。